

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 7

по курсу «Операционные системы»

на тему:

«Буферизованный и небуферизованный ввод-вывод»

Студент ИУ7-62Б (Группа) _____ (Подпись, дата) Жаринов М. А. (И. О. Фамилия)

Преподаватель _____ (Подпись, дата) Рязанова Н. Ю. (И. О. Фамилия)

2026 г.

1 Структуры

```
struct _IO_FILE
{
    int _flags;     /* High-order word is _IO_MAGIC; rest is flags. */

    /* The following pointers correspond to the C++ streambuf
       protocol. */
    char *_IO_read_ptr; /* Current read pointer */
    char *_IO_read_end; /* End of get area. */
    char *_IO_read_base;    /* Start of putback+get area. */
    char *_IO_write_base;   /* Start of put area. */
    char *_IO_write_ptr;    /* Current put pointer. */
    char *_IO_write_end;    /* End of put area. */
    char *_IO_buf_base; /* Start of reserve area. */
    char *_IO_buf_end;  /* End of reserve area. */

    /* The following fields are used to support backing up and undo. */
    char *_IO_save_base; /* Pointer to start of non-current get area.
       */
    char *_IO_backup_base;  /* Pointer to first valid character of
       backup area */
    char *_IO_save_end; /* Pointer to end of non-current get area. */

    struct _IO_marker *_markers;

    struct _IO_FILE *_chain;

    int _fileno;
    int _flags2:24;
    /* Fallback buffer to use when malloc fails to allocate one.  */
    char _short_backupbuf[1];
    __off_t _old_offset; /* This used to be _offset but it's too
       small.  */

    /* 1+column number of pbase(); 0 is unknown. */
    unsigned short _cur_column;
    signed char _vtable_offset;
    char _shortbuf[1];

    _IO_lock_t *_lock;
    __off64_t _offset;
    /* Wide character stream stuff.  */
    struct _IO_codecvt *_codecvt;
    struct _IO_wide_data *_wide_data;
    struct _IO_FILE *_freeres_list;
    void *_freeres_buf;
```

```
    struct _IO_FILE **_prevchain;
    int _mode;
    int _unused3;
    __uint64_t _total_written;
    /* Make sure we don't get into trouble again.  */
    char _unused2[12 * sizeof (int) - 5 * sizeof (void *)];
};
```

Листинг 1: Структура _IO_FILE

```
struct stat
{
    __dev_t st_dev;     /* Device.  */
    __ino_t st_ino;     /* File serial number.  */
    __nlink_t st_nlink;     /* Link count.  */
    __mode_t st_mode;       /* File mode.  */
    __uid_t st_uid;     /* User ID of the file's owner. */
    __gid_t st_gid;     /* Group ID of the file's group.*/
    int __pad0;
    __dev_t st_rdev;        /* Device number, if device.  */
    __off_t st_size;            /* Size of file, in bytes.  */
    __blksize_t st_blksize; /* Optimal block size for I/O.  */
    __blkcnt_t st_blocks;       /* Number 512-byte blocks allocated. */
    # ifdef __USE_XOPEN2K8
    /* Nanosecond resolution timestamps are stored in a format
    equivalent to 'struct timespec'.  This is the type used
    whenever possible but the Unix namespace rules do not allow the
    identifier 'timespec' to appear in the <sys/stat.h> header.
    Therefore we have to handle the use of this header in strictly
    standard-compliant sources special.  */
    struct timespec st_atim;        /* Time of last access.  */
    struct timespec st_mtim;        /* Time of last modification.  */
    struct timespec st_ctim;        /* Time of last status change.  */
    #  define st_atime st_atim.tv_sec   /* Backward compatibility.  */
    #  define st_mtime st_mtim.tv_sec
    #  define st_ctime st_ctim.tv_sec
    # else
    __time_t st_atime;          /* Time of last access.  */
    __syscall_ulong_t st_atimensec; /* Nscecs of last access.  */
    __time_t st_mtime;          /* Time of last modification.  */
    __syscall_ulong_t st_mtimensec; /* Nsecs of last modification.  */
    __time_t st_ctime;          /* Time of last status change.  */
    __syscall_ulong_t st_ctimensec; /* Nsecs of last status change.  */
    # endif
    __syscall_slong_t __glibc_reserved[3];
};
```

Листинг 2: Структура stat

2 Программы

2.1 Программа 1

```c
#include <fcntl.h>
#include <stdio.h>

int main() {
        FILE *fs1, *fs2;
        char buff1[20], buff2[20], c;
        int flag1 = 1, flag2 = 1;
        int fd = open("alphabet.txt", O_RDONLY);
        fs1 = fdopen(fd, "r");
        setvbuf(fs1, buff1, _IOFBF, 20);
        fs2 = fdopen(fd, "r");
        setvbuf(fs2, buff2, _IOFBF, 20);
        while (flag1 == 1 || flag2 == 1) {
                flag1 = fscanf(fs1, "%c", &c);
                if (flag1 == 1)
                fprintf(stdout, "%c", c);
                flag2 = fscanf(fs2, "%c", &c);
                if (flag2 == 1)
                fprintf(stdout, "%c", c);
            }
        fprintf(stdout, "\n");
        return 0;
    }
```

Листинг 3: Исходный код Программы 1

Скриншот результата выполнения программы 1 представлен на рисунке 1.

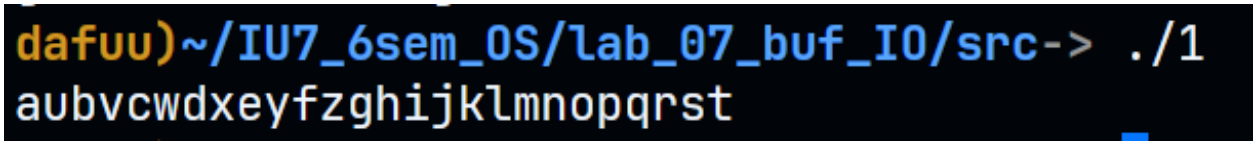


Рисунок 1 – Результат выполнения программы 1

Анализ полученного результата

При запуске программы системный вызов open() открывает файл alphabet.txt в режиме O_RDONLY, в результате чего в пространстве ядра формируется структура struct file. Ссылка на неё заносится в таблицу файловых дескрипторов текущего процесса под индексом 3, поскольку младшие дескрипторы 0, 1 и 2 по умолчанию закреплены за стандартными потоками stdin, stdout и stderr. Этот индекс возвращается в качестве значения переменной fd.

Далее библиотечная функция fdopen() связывает дескриптор fd с двумя логическими потоками fs1 и fs2, создавая для каждого структуру FILE (фактически struct _IO_FILE), в которой поле _fileno принимает значение fd. С помощью функции setvbuf() для обоих потоков fs1 и fs2 устанавливается режим полной буферизации (_IOFBF) со своим пользовательским буфером размером 20 байт (buff1 и buff2 соответственно).

Первый вызов fscanf() для fs1 приводит к заполнению буфера buff1 первыми 20 байтами файла (символами от a до t). В системной структуре file при этом сдвигается указатель текущей позиции f_pos на 20. Из буфера buff1 извлекается символ a, сохраняется в c и выводится на экран, а локальный указатель чтения _IO_read_ptr для fs1 перемещается на один байт вперед.

Поскольку потоки fs1 и fs2 используют общий файловый дескриптор, они разделяют одну структуру file и текущую позицию f_pos. Поэтому при первом вызове fscanf() для fs2 чтение в буфер buff2 начинается с позиции f_pos = 20. В буфер buff2 считываются оставшиеся 6 символов файла (от u до z), а f_pos увеличивается до конца файла. Символ u извлекается из buff2 и записывается в стандартный вывод.

В последующих шагах цикла символы попеременно считываются из уже заполненных буферов buff1 и buff2. После того как буфер buff2 будет полностью исчерпан (когда _IO_read_ptr сравняется с _IO_read_end), программа продолжит считывать и выводить оставшиеся символы исключительно из буфера buff1 (вплоть до t), пока и в нём не закончатся данные.

На рисунке 2 отображены связи структур для программы 1.

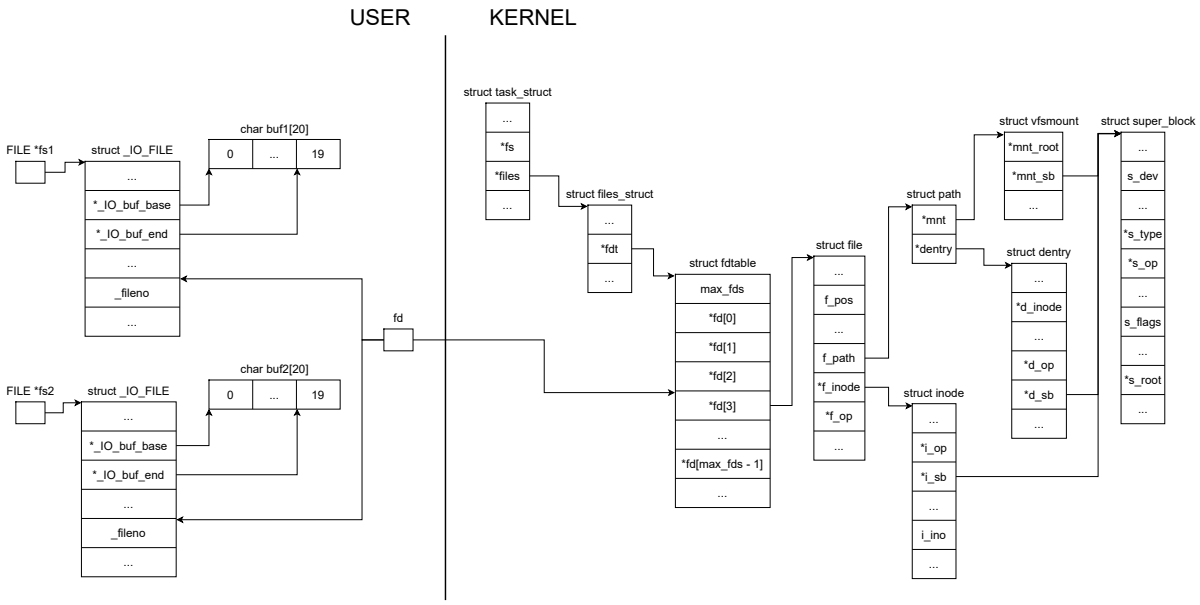


Рисунок 2 – Схема связи структур для программы 1

2.2 Программа 2

2.2.1 Один поток

```
#include <fcntl.h>
#include <unistd.h>

int main() {
    char c;
    int flag1 = 1, flag2 = 1;
    int fd1 = open("alphabet.txt", O_RDONLY);
    int fd2 = open("alphabet.txt", O_RDONLY);
    while (flag1 == 1 && flag2 == 1) {
        flag1 = read(fd1, &c, 1);
        if (flag1 == 1) {
            write(1, &c, 1);
            flag2 = read(fd2, &c, 1);
            if (flag2 == 1)
                write(1, &c, 1);
        }
    }
    close(fd1);
    close(fd2);
    c = '\n';
    write(1, &c, 1);
    return 0;
}
```

Листинг 4: Исходный код однопоточной Программы 2

Скриншот результата выполнения однопоточной программы 2 представлен на рисунке 3.

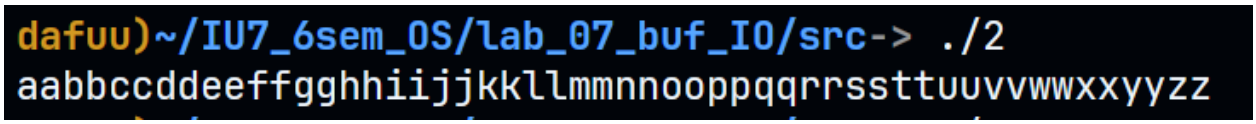


Рисунок 3 – Результат выполнения однопоточной программы 2

Анализ полученного результата

С помощью двух системных вызовов open() программа открывает файл alphabet.txt в режиме O_RDONLY, возвращая дескрипторы fd1 и fd2. В таблице файлов процесса под номерами 3 и 4 (поскольку дескрипторы 0, 1 и 2 заняты стандартными потоками ввода-вывода) создаются записи, указывающие на две независимые системные структуры struct file в ядре. Обе эти структуры указывают на общий inode одного и того же файла.

Каждая структура struct file обладает собственным указателем текущего смещения f_pos, который изначально равен нулю. Системный вызов read() для fd1 считывает из файла один символ (символ a), сдвигая f_pos в первой структуре file на 1, после чего символ отправляется в стандартный вывод через write().

Второй вызов read() использует дескриптор fd2, у которого f_pos во второй структуре file по-прежнему равен 0. Из-за этого из файла считывается тот же самый символ a, и f_pos во второй структуре увеличивается на 1.

Процесс поочередного чтения через fd1 и fd2 продолжается в цикле до тех пор, пока f_pos в обеих структурах не достигнет конца файла (размера в 26 байт). Благодаря независимости файловых позиций, каждый символ последовательно читается и выводится по два раза, что дает строго детерминированный результат: aabbcc...zz.

2.2.2 Один основной поток и один дополнительный поток

```
#include <fcntl.h>
#include <unistd.h>
#include <pthread.h>

int flag1 = 1, flag2 = 1;

void* thread1_func(void* arg) {
    char c;
    int fd = *(int*)arg;
    while (flag1 == 1 && flag2 == 1) {
        flag1 = read(fd, &c, 1);
        if (flag1 == 1) {
            write(1, &c, 1);
        }
    }
    close(fd);
    return NULL;
}

int main() {
    char c;
    pthread_t t1, t2;
    int fd1 = open("alphabet.txt", O_RDONLY);
    int fd2 = open("alphabet.txt", O_RDONLY);
    pthread_create(&t1, NULL, thread1_func, &fd1);
    while (flag1 == 1 && flag2 == 1) {
        flag2 = read(fd2, &c, 1);
        if (flag2 == 1) {
            write(1, &c, 1);
        }
    }
    close(fd2);
    pthread_join(t1, NULL);
    c = '\n';
    write(1, &c, 1);
    return 0;
}
```

Листинг 5: Исходный код Программы 2 с одним дополнительным потоком

Скриншот результата выполнения программы 2 с одним дополнительным потоком представлен на рисунке 4.

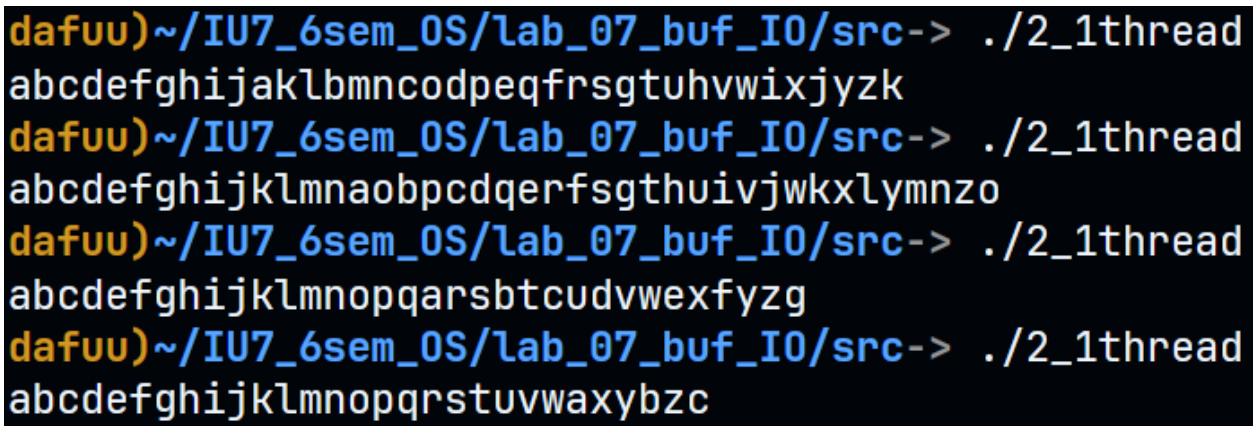


Рисунок 4 – Результат выполнения программы 2 с одним дополнительным потоком

Анализ полученного результата

В данном варианте главный поток с помощью pthread_create() порождает дополнительный поток исполнения, передавая ему указатель на дескриптор fd1. Процедура создания нового потока операционной системой занимает некоторое время. В этот промежуток главный поток, выполняющий чтение через fd2, может успеть прочесть и вывести на экран как часть файла, так и весь его объем целиком.

После того как порожденный поток переходит в активное состояние, чтение данных через независимые файловые дескрипторы и их вывод производятся обоими потоками параллельно. Очередность выполнения их операций ввода-вывода полностью определяется планировщиком операционной системы.

Каждый символ из файла будет гарантированно прочитан и выведен дважды (по одному разу каждым из потоков), но их взаимное расположение в выходной строке недетерминировано и зависит от особенностей распределения квантов процессорного времени между потоками.

2.2.3 Два дополнительных потока

```
#include <fcntl.h>
#include <unistd.h>
#include <pthread.h>

int flag1 = 1, flag2 = 1;

void* thread1_func(void* arg) {
    char c;
    int fd = *(int*)arg;
    while (flag1 == 1 && flag2 == 1) {
        flag1 = read(fd, &c, 1);
        if (flag1 == 1) {
            write(1, &c, 1);
        }
    }
    close(fd);
    return NULL;
}

void* thread2_func(void* arg) {
    char c;
    int fd = *(int*)arg;
    while (flag1 == 1 && flag2 == 1) {
        flag2 = read(fd, &c, 1);
        if (flag2 == 1) {
            write(1, &c, 1);
        }
    }
    close(fd);
    return NULL;
}

int main() {
    char c = '\n';
    pthread_t t1, t2;
    int fd1 = open("alphabet.txt", O_RDONLY);
    int fd2 = open("alphabet.txt", O_RDONLY);
    pthread_create(&t1, NULL, thread1_func, &fd1);
    pthread_create(&t2, NULL, thread2_func, &fd2);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    write(1, &c, 1);
    return 0;
}
```

Листинг 6: Исходный код Программы 2 с двумя дополнительными потоками

Скриншот результата выполнения программы 2 с двумя дополнительными потоками представлен на рисунке 5.

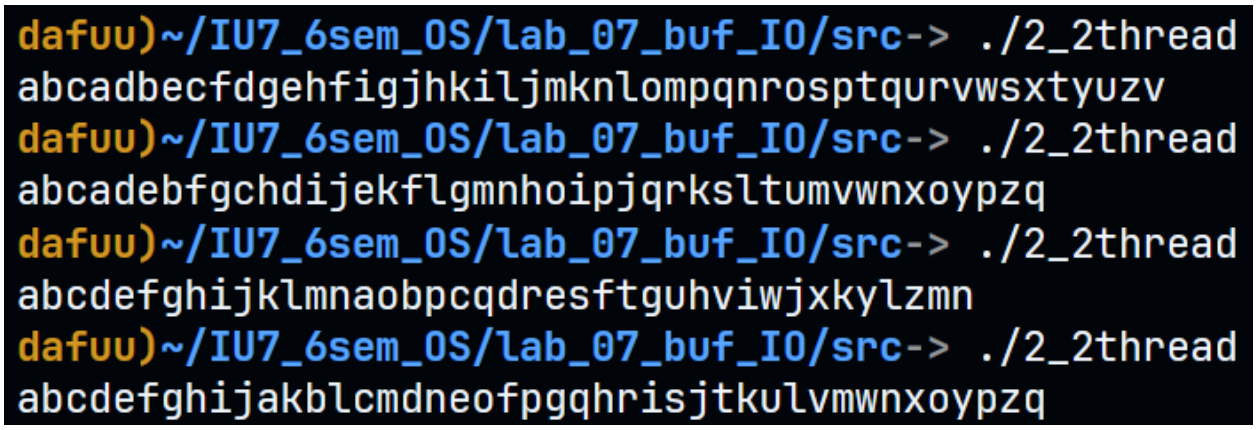


Рисунок 5 – Результат выполнения программы 2 с двумя дополнительными потоками

Анализ полученного результата

Главный поток создает два дочерних потока с помощью последовательных вызовов pthread_create(), передавая первому указатель на fd1, а второму — на fd2. Сам главный поток после этого блокируется на вызовах pthread_join(), дожидаясь их завершения.

Инициализация и запуск потоков требуют системного времени, и порядок, в котором они приступят к выполнению своих функций, заранее не предопределен. Логика планировщика задач операционной системы полностью управляет распределением ресурсов между этими потоками.

В результате оба потока параллельно читают весь файл с самого начала (поскольку структуры struct file в ядре у них независимы и имеют собственные смещения f_pos), выводя символы в общий стандартный поток. Итоговая строка содержит по два каждого символа, но их последовательность носит случайный характер.

На рисунке 6 отображены связи структур для программы 2.

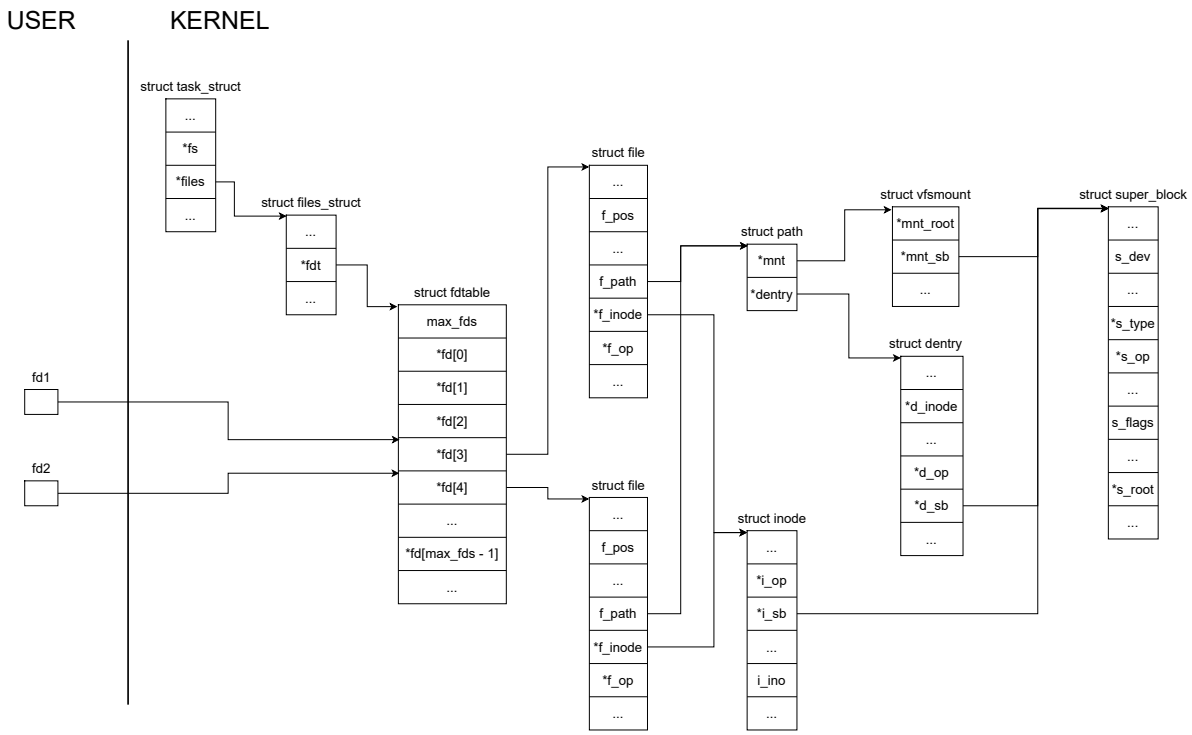


Рисунок 6 – Схема связи структур для программы 2

2.3 Программа 3

2.3.1 Небуферизованный ввод-вывод

```
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <stdio.h>

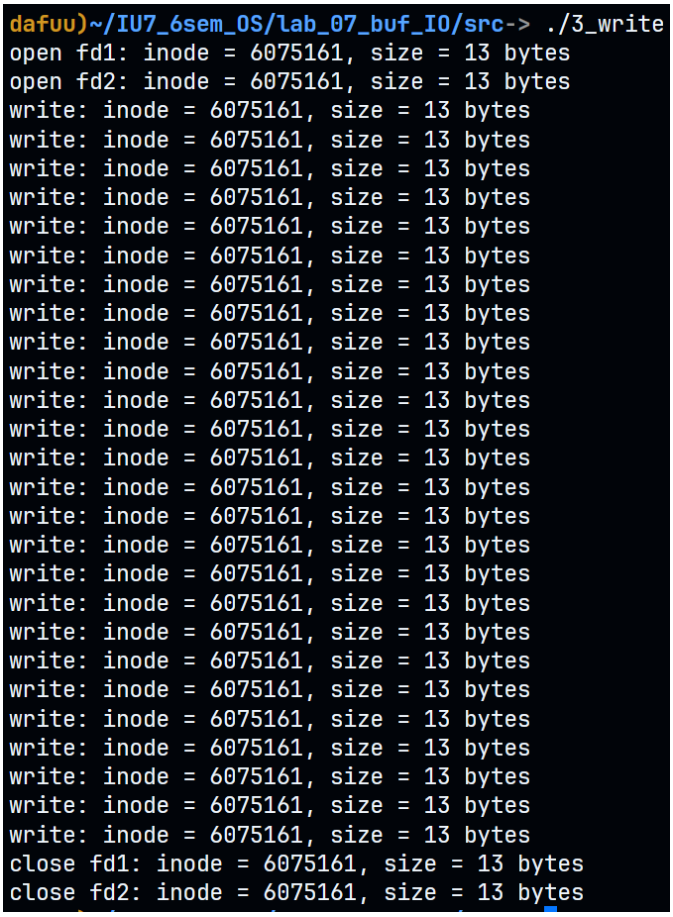
void print_stat(char *msg) {
    struct stat statbuf;
    stat("q.txt", &statbuf);
    fprintf(stdout, "%s: inode = %ld, size = %ld bytes\n", msg,
        statbuf.st_ino, statbuf.st_size);
}

int main() {
    int fd1 = open("q.txt", O_RDWR);
    print_stat("open fd1");
    int fd2 = open("q.txt", O_RDWR);
    print_stat("open fd2");
    for (char c = 'a'; c <= 'z'; c++) {
        if (c % 2)
            write(fd1, &c, 1);
```

```
20          else
21              write(fd2, &c, 1);
22          print_stat("write");
23      }
24      close(fd1);
25      print_stat("close fd1");
26      close(fd2);
27      print_stat("close fd2");
28      return 0;
29  }
```

Листинг 7: Исходный код Программы 3 с небуферизованным вводом-выводом

Скриншот результата выполнения программы 3 с небуферизованным вводом-выводом представлен на рисунке 7.


dafuu)~/IU7_6sem_OS/lab_07_buf_IO/src-> ./3_write
open fd1: inode = 6075161, size = 13 bytes
open fd2: inode = 6075161, size = 13 bytes
write: inode = 6075161, size = 13 bytes
write: inode = 6075161, size = 13 bytes
write

Анализ полученного результата

При выполнении двух системных вызовов open() с флагом O_RDWR (в режиме «чтения - записи») создаются два файловых дескриптора (fd1 и fd2) с номерами 3 и 4. Соответствующие им независимые структуры struct file в ядре указывают на один inode файла q.txt, но при этом обладают разными значениями текущей позиции f_pos, изначально равными нулю.

На первом шаге цикла вызов write() для fd1 записывает символ a в начало файла, сдвигая f_pos в первой структуре file на 1. Текущий размер q.txt становится равным 1 байту.

На втором шаге цикла выполняется запись через дескриптор fd2. Поскольку f_pos во второй структуре file все еще равен 0, системный вызов write() перезаписывает ранее сохраненный символ a символом b. Размер файла при этом не увеличивается и остается равным 1 байту, а f_pos для fd2 сдвигается на 1.

Подобное поочередное перекрытие символов продолжается на протяжении всей работы цикла. Символы, записываемые через fd1, заменяются символами, отправляемыми через fd2.

В результате в q.txt сохраняются только четные буквы алфавита (b, d, f...), начиная с b, а финальный размер файла составляет 13 байт (половина алфавита).

2.3.2 Буферизованный ввод-вывод

```
1  #include <fcntl.h>
2  #include <stdio.h>
3  #include <sys/stat.h>
4
5  void print_stat(char *msg) {
6      struct stat statbuf;
7      stat("q.txt", &statbuf);
8      fprintf(stdout, "%s: inode = %ld, size = %ld bytes\n", msg,
           statbuf.st_ino, statbuf.st_size);
9  }
10
11 int main() {
12     FILE *fs1 = fopen("q.txt", "w");
13     print_stat("fopen fs1");
14     FILE *fs2 = fopen("q.txt", "w");
15     print_stat("fopen fs2");
16     for (char c = 'a'; c <= 'z'; c++) {
```

```
17          if (c % 2)
18              fprintf(fs1, "%c", c);
19          else
20              fprintf(fs2, "%c", c);
21          print_stat("fprintf");
22      }
23      fclose(fs1);
24      print_stat("fclose fs1");
25      fclose(fs2);
26      print_stat("fclose fs2");
27      return 0;
28  }
```

Листинг 8: Исходный код Программы 3 с буферизованным вводом-выводом

Скриншот результата выполнения программы 3 с буферизованным вводом-выводом представлен на рисунке 8.

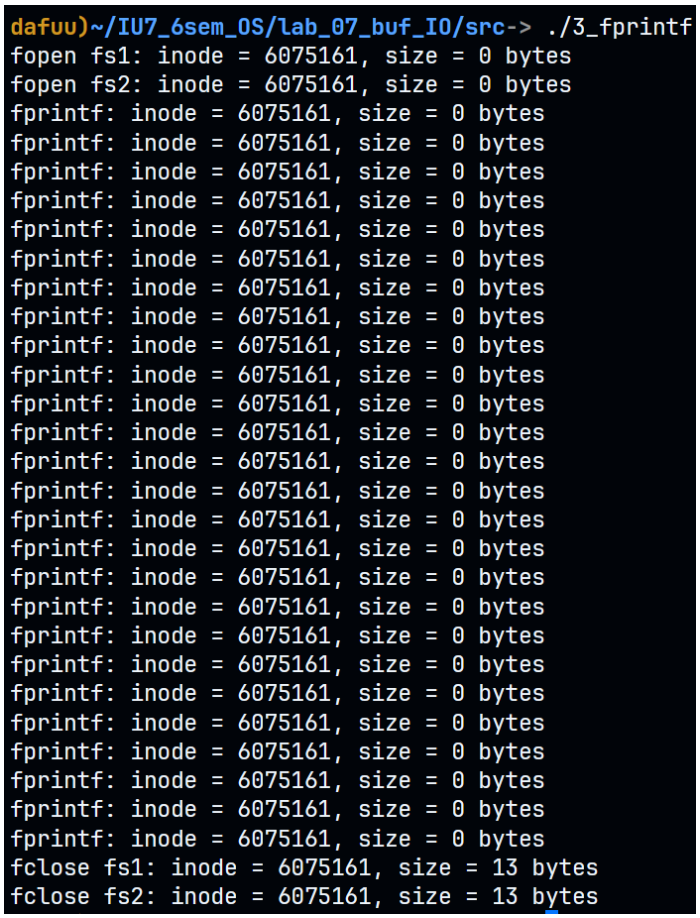


Рисунок 8 – Результат выполнения программы 3 с буферизованным вводом-выводом

Анализ полученного результата

Функция fopen() в режиме "w" открывает файл q.txt дважды, возвращая указатели на две структуры FILE (fs1 и fs2), ассоциированные с независимыми дескрипторами 3 и 4. В пространстве ядра для них создаются две структуры struct file, указывающие на один inode. При этом библиотека stdio.h выделяет в пользовательском пространстве для каждого потока собственный буфер размером 4096 байт.

При выполнении цикла вызовы fprintf() записывают данные исключительно в эти локальные буферы, не выполняя немедленных системных вызовов записи в ядро. В буфере fs1 накапливаются нечетные символы алфавита (начиная с a), а в буфере fs2 — четные (начиная с b). Соответствующие указатели _IO_WRITE_PTR смещаются в памяти на 13 байт.

Запись данных на диск происходит при вызове fclose(). Сначала закрывается поток fs1. Накопленные 13 байт из буфера fs1 записываются в файл q.txt с нулевой позиции (так как f_pos дескриптора 3 равен 0). Размер файла становится равен 13 байтам, а f_pos для первого дескриптора увеличивается до 13.

Затем вызывается fclose() для fs2. Поскольку позиция f_pos во второй структуре file (соответствующей дескриптору 4) все еще равна 0, 13 байт из буфера fs2 записываются также с начала файла. Это перезаписывает данные, ранее сохраненные из fs1. Итоговый размер файла q.txt остается равным 13 байтам, но его содержимое теперь состоит только из символов буфера fs2 (bdfhjlnprtvxz). Таким образом, результат зависит от порядка вызова функций закрытия потоков.

На рисунке 9 отображены связи структур для программы 3 с буферизованным вводом-выводом.

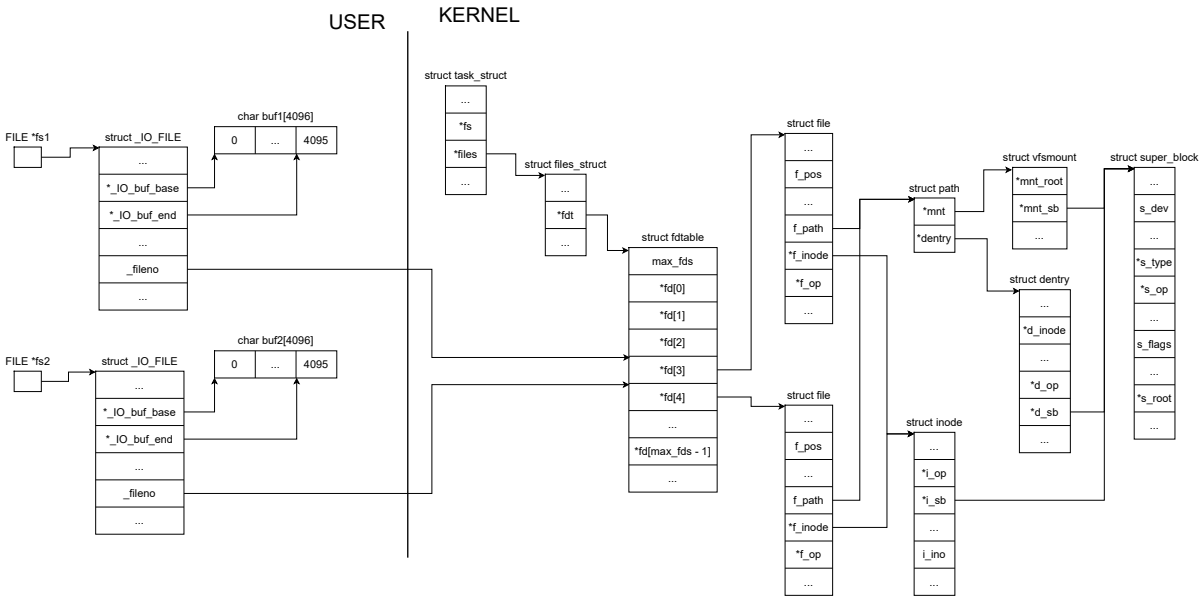


Рисунок 9 – Схема связи структур для программы 3 с буферизованным вводом-выводом

2.3.3 Вывод

Для предотвращения потери и взаимной перезаписи данных при работе с одним файлом через несколько независимых дескрипторов необходимо использовать режим дозаписи (append):

1. В небуферизованном вводе-выводе (системный вызов open) следует использовать флаг `O_APPEND` (в сочетании с `O_WRONLY` или `O_RDWR`). Данный флаг гарантирует смещение указателя текущей позиции ядра (`f_pos`) в конец файла перед каждым вызовом `write()`, что исключает перезапись ранее записанных данных другими дескрипторами.

2. В буферизованном вводе-выводе (функция fopen) файл необходимо открывать в режиме `"a"` или `"a+"`. Этот режим на уровне ядра добавляет флаг `O_APPEND`. При вызове функций очистки и закрытия потоков (`fflush` или `fclose`) данные из пользовательских буферов будут гарантированно дописаны в конец файла, а не записаны поверх нового содержимого.